

**CENTRO PAULA SOUZA
FACULDADE DE TECNOLOGIA DE VOTORANTIM
CURSO SUPERIOR DE TECNOLOGIA EM CIÊNCIA DE DADOS PARA NEGÓCIOS**

**PROJETO INTEGRADOR VI
SISTEMA DE RECOMENDAÇÃO POR AFINIDADE (BERTO
FERRAMENTAS)**

**ANEXO B: GUIA DE IMPLANTAÇÃO E OPERAÇÃO EM NUVEM (DEPLOY
GUIDE)**

**ERIC BERTO
MICHEL CIRILO**

**VOTORANTIM / SP
2026**

IDENTIFICAÇÃO DO DOCUMENTO TÉCNICO

Tabela 1 – Metadados do Guia de Deploy e Operação (PI6)

Projeto Integrador:	PI6 — Projetos em Ciência de Dados II (6º Semestre)
Empresa Parceira:	Berto Ferramentas (Sorocaba / SP)
Autores & Equipe:	Eric Berto & Michel Cirilo (Fatec Votorantim)
Plataforma de Deploy:	Databricks Workspace (AWS) • Databricks Apps (dev-pi-berto)
Tipo de Entregável:	Anexo B — Guia de Implantação e Operação em Nuvem (Deploy Guide)

Fonte: Elaboração própria dos autores (2026).

 **Evolução Arquitetural (PI5 -> PI6):** No PI5 foi desenvolvido um protótipo preliminar em ambiente local. No PI6, a solução foi totalmente elevada para o ambiente industrial em nuvem no Databricks Apps, escalando a mineração de regras de associação de 2.004 para 185.104 regras validadas. O ecossistema PI6 integra governança centralizada no Databricks Unity Catalog, rastreamento MLOps no MLflow Experiments e containerização automatizada da interface web em Streamlit (v3.2).

1. OBJETIVO E ESCOPO DE IMPLANTAÇÃO (PI6)

Este documento estabelece o procedimento técnico oficial de implantação, operação, governança e manutenção do Sistema de Recomendação por Afinidade do Projeto Integrador VI (PI6) desenvolvido para a empresa Berto Ferramentas. O escopo deste guia é estritamente focado na arquitetura de nuvem em produção implementada no ecossistema Databricks na AWS, eliminando dependências de servidores locais legados.

2. ARQUITETURA DA SOLUÇÃO EM NUVEM

A infraestrutura do PI6 opera sob um modelo cloud-native desacoplado e altamente performático, estruturado em quatro camadas principais:

- **Camada de Dados & Governança (Unity Catalog):** Armazenamento centralizado no schema workspace.bertoFerramentas composto por 7 tabelas Delta gerenciadas que garantem rastreabilidade, linhagem e controle de acesso.
- **Camada de Processamento Analítico & PLN:** Notebooks Databricks executando carga da base Raw (1_catalogo.ipynb) e limpeza semântica via Regex e RapidFuzz (2_Sistema_Recomendacao_Clean.ipynb, token_sort_ratio,

threshold=83), reduzindo 13.068 SKUs despadronizados para 3.241 Entidades Mestres.

- **Camada de Modelagem Preditiva (FP-Growth + MLflow):** Execução distribuída do algoritmo FP-Growth (suporte 0,0002 e lift min 1,3), extraindo 185.104 regras validadas com versionamento de hiperparâmetros no MLflow Experiments.
- **Camada de Aplicação & Apresentação (Databricks Apps):** Serviço gerenciado de microserviço web em Streamlit v3.2 (dev-pi-berto) rodando em contêiner dedicado na AWS com busca por autocomplete e descompactação de estoque físico.

3. ORGANIZAÇÃO DO WORKSPACE DATABRICKS (DATABRICKSBERTO)

Os artefatos do projeto estão organizados no diretório gerenciado /Workspace/Users/michelcirilo@hotmail.com/databricks_apps/Berto_PI6/DatabricksBerto, seguindo rigorosamente a padronização institucional do Manual do PI6:

Tabela 2 – Estrutura do Repositório Git e Workspace Databricks (DatabricksBerto)

Diretório / Arquivo	Tipo	Descrição Técnica & Função no PI6
experiments/	Pasta Workspace	Rastreamento e registros de corridas do MLflow Experiments (FP_Growth_Recomendacao).
notebooks/	Pasta Workspace	Cadernos Databricks de ETL e IA: '1_catalogo.ipynb' (Carga/Atualização da Base Raw) e '2_Sistema_Recomendacao_Clean.ipynb' (PLN, FP-Growth e exportação).
src/	Pasta Workspace	Scripts Python modulares contendo pipelines de transformação e rotinas de apoio.
streamlit-app/	Pasta Workspace	Código-fonte da app web (app.py), arquivos de deploy (app.yaml) e artefatos de dados (data/).
tests/	Pasta Workspace	Testes automatizados unitários, validação de schema Delta e testes de integridade.
README.md	Arquivo Raiz	Documentação executiva, manual de instruções e arquitetura geral do repositório.

Fonte: Autores (2026) / Databricks Workspace.

4. GOVERNANÇA DE DADOS NO UNITY CATALOG

A gestão centralizada dos dados transacionais da Berto Ferramentas é realizada no schema workspace.bertoFerramentas do Databricks Catalog Explorer. O

ecossistema contém 7 tabelas Delta com controle rigoroso de permissões e linhagem:

Tabela 3 – Catálogo de Tabelas Delta Gerenciadas no Unity Catalog (workspace.bertoFerramentas)

Tabela Delta (Unity Catalog)	Função Técnica no Pipeline de IA / Origem dos Dados
tab_venda	Tabela de cabeçalho com metadados das transações de venda da loja (Base Raw).
tab_venda_item	Tabela de itens transacionados (preços, quantidades, custos e descontos) (Base Raw).
tab_produto	Tabela dimensional de cadastro bruto de produtos transacionados (Base Raw).
input_ia_tab_produto_processado	Catálogo higienizado após etapas de limpeza por Regex e PLN.
input_ia_tab_vinculo_mestre	Tabela de ligação De-Para entre SKUs físicos e Entidades Mestres.
input_ia_dataset_produto_mestre	Dataset agrupado de produtos mestres higienizados para treinamento.
input_ia_recomendacao	Tabela Delta final contendo as 185.104 regras de associação para consulta.

Fonte: Autores (2026) / Databricks Catalog Explorer.

5. PROCEDIMENTO DE DEPLOY DA APLICAÇÃO (DEV-PI-BERTO)

A disponibilização da interface web interativa do cliente ocorre via Databricks Apps, um serviço totalmente gerenciado na nuvem AWS. O microserviço recebeu o identificador dev-pi-berto e é publicado no endpoint seguro: <https://dev-pi-berto-7474644370532329.aws.databricksapps.com>

5.1 Artefatos do Front-End (streamlit-app/)

A pasta streamlit-app/ contém todos os componentes necessários para a compilação do contêiner:

- **app.py:** Script principal em Streamlit v3.2 implementando busca rápida autocomplete, digitação livre, motor de recomendação e descompactação de estoque físico.

- **app.yaml & manifest.yaml:** Arquivos de especificação do Databricks Apps contendo comandos de entrada (streamlit run app.py) e recursos computacionais.
- **requirements.txt:** Declaração de dependências do Python (streamlit, pandas, pyarrow, numpy, rapidfuzz).
- **data/db_conhecimento.parquet:** Base colunar contendo as 185.104 regras de associação geradas pelo FP-Growth.
- **data/catalogo_busca.parquet:** Mapeamento entre nomes higienizados e Entidades Mestres.
- **data/catalogo_detalhado.pkl:** Dicionário Pickle serializado para engenharia reversa do estoque físico.
- **data/metadata.json:** Metadados de auditoria (data da última atualização, versão do modelo e estatísticas).

5.2 Passo a Passo para Deploy e Atualização

Passo 1 — Carga e Atualização da Base Raw: Executar o notebook '1_catalogo.ipynb' no Databricks. Este script processa as fontes transacionais brutas da Berto Ferramentas e atualiza a camada de dados Raw no Unity Catalog (schema workspace.bertoFerramentas, garantindo que novos registros das tabelas 'tab_venda', 'tab_venda_item' e 'tab_produto' estejam prontos para processamento).

Passo 2 — Processamento Semântico e Geração de Artefatos de IA: Executar o notebook '2_Sistema_Recomendacao_Clean.ipynb' no Databricks. O script realiza a higienização por Regex e PLN/RapidFuzz, executa a mineração de regras de associação via FP-Growth, salva as tabelas Delta processadas no Unity Catalog e exporta os arquivos otimizados Parquet e Pickle para o diretório 'streamlit-app/data/'.

Passo 3 — Sincronização do Auto-Setup: A aplicação 'app.py' executa uma rotina automática que valida a integridade dos arquivos Parquet/Pickle e instancia os dicionários em memória usando o decorador @st.cache_resource.

Passo 4 — Deploy via Databricks Apps: Acesse a aba 'Compute' -> 'Apps' no Databricks Workspace, selecione 'dev-pi-berto' e clique em 'Deploy'. O serviço realiza o download do código atualizado, instala dependências do 'requirements.txt', constrói a imagem e inicia o contêiner.

Passo 5 — Validação de Status: Confirme se o status exibe 'Running' com Compute Status 'Active' (Configuração Medium: 2 vCPUs, 6 GB RAM, 0.5

DBU/hora). Acesse a URL pública para homologação final da aplicação e testes de recomendação no balcão.

6. MONITORAMENTO E PLANO DE MANUTENÇÃO (MLOPS)

Para garantir a sustentabilidade operacional e a acurácia do motor de recomendação ao longo do tempo, a equipe da Berto Ferramentas deve seguir os protocolos de governança MLOps:

- **Rastreabilidade via MLflow:** Todas as execuções de retreinamento são registradas no Databricks Experiments ('FP_Growth_Recomendacao'). Hiperparâmetros de suporte (0.0002) e lift (1.3) são salvos junto com o grafo das 185.104 regras.
- **Plano de Retreinamento Trimestral:** A cada 90 dias, a esteira sequencial deve ser reexecutada: primeiro o notebook '1_catalogo.ipynb' para ingestão das novas vendas na base Raw, seguido do notebook '2_Sistema_Recomendacao_Clean.ipynb' para re-gerar as regras de associação e atualizar os artefatos Parquet no Streamlit.
- **Monitoramento de Recursos e Logs:** Acompanhamento contínuo dos logs do contêiner e tempo de resposta através do painel 'Logs' e 'Insights' do Databricks Apps.

7. TABELA DE SOLUÇÃO DE PROBLEMAS (TROUBLESHOOTING)

Tabela 4 – Matriz de Solução de Problemas e Diagnóstico Operacional

Sintoma / Falha	Causa Provável	Ação Corretiva Recomendada
Aplicação exibe erro ao carregar recomendações	Arquivo db_conhecimento.parquet desatualizado ou corrompido.	Executar em ordem os notebooks '1_catalogo.ipynb' e '2_Sistema_Recomendacao_Clean.ipynb' no Databricks e forçar o redeploy da app.
App fora do ar (HTTP 502 / Build Error)	Falha de instalação de pacotes no requirements.txt ou estouro de recursos.	Verificar a aba 'Logs' no Databricks Apps, validar sintaxe das dependências e reiniciar a instância computacional.
Produto recém-cadastrado sem sugestões	Item de cauda longa sem transações na base Raw ou pendente de carga.	Executar '1_catalogo.ipynb' para atualizar a base Raw e aguardar a reexecução do pipeline de IA no retreinamento trimestral.

Fonte: Autores (2026) / Suporte de Operação Berto Ferramentas.